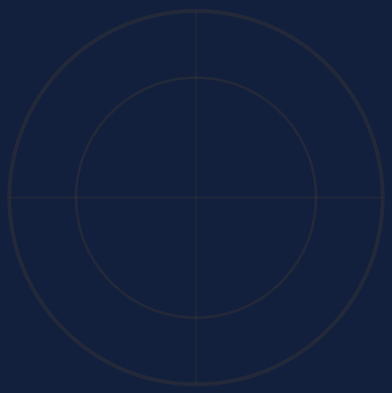
# React-PDF
# + AI

The builder's guide to premium PDF generation.
Practical patterns for developers who ship.

LANDON MILES



landonmiles.com

2026

# Contents

FOUNDATIONS



DESIGN SYSTEM



CRAFT



SHIPPING

CHAPTER 01

# Introduction

Why react-pdf and AI belong together – and what this book will teach you.

## What This Book Is

A practical guide for developers using AI coding agents – Claude Code, Cursor, Copilot – to generate PDFs with @react-pdf/renderer. You know React. What you may not know is how to structure a react-pdf project so AI produces premium output instead of generic templates. That gap is what this book closes.

Every page you're reading was built with these patterns. The theme file, component library, build scripts, and reference docs are the actual tools that produced this document – not hypothetical examples.

### Built for AI-Assisted Development

The patterns in this book turn your AI coding agent from a generic template generator into a precision PDF builder.

## The Core Thesis

React-PDF is uniquely suited for AI-assisted development. It uses JSX – which AI models have deep training data on. It uses flexbox for layout – a familiar mental model. It renders server-side, so you can automate the entire pipeline.

However, the default output from an LLM rarely meets professional standards. Closing that gap is what the rest of this book is about – the specific architectural and design patterns that turn generic output into something premium.

## Who This Is For

This book is for developers who want to move beyond "Hello World" PDFs. If you're building invoices, resumes, reports, or ebooks, and you want to use AI to speed up the process without sacrificing quality, this is for you.

We assume you know React and basic CSS. We don't assume you're an expert in PDF typography or the specific quirks of the react-pdf engine – we'll cover those as we go.

## Why React-PDF?

While there are many ways to generate PDFs (Puppeteer, LaTeX, specialized APIs), @react-pdf/renderer offers the best balance of developer experience and AI compatibility. Because it's "just React," your existing skills – and your AI's knowledge of React patterns – apply directly.

| Approach | Trade-off | AI fit |
|---|---|---|
| Puppeteer | Ships a headless browser; heavy and slow | Medium |
| LaTeX | Beautiful output, steep and arcane syntax | Low |
| PDF APIs | Fast to start, opaque and hard to customize | Low |
| React-PDF | Component model you already think in | High |

### Why AI fit matters

Your AI agent has seen millions of React components. When the layout is JSX, it can reason about structure, reuse your design tokens, and edit a page the same way it edits any component – no PDF-specific dialect to learn first.

CHAPTER 02

# React-PDF Fundamentals

Components, styling, layout, and the rules of the rendering engine.

# Component Hierarchy

Every react-pdf document has a required root and page structure. Inside a Page, supported react-pdf primitives can be direct children. Invalid trees may warn, omit content, or fail to render.

```tsx
import { Document, Page, View, Text, Image }
from '@react-pdf/renderer';

const MyDoc = () => (
<Document>
<Page size="LETTER">
<View>
<Text>Ordinary visible copy belongs in Text.</Text>
<Image src="photo.png" />
</View>
</Page>
</Document>
);
```

The rules:

- Document is the root – its rendered direct children must be Page elements
- Page defines a physical page – size, orientation, margins
- View is an optional container (like div) – it handles layout via flexbox
- Text renders ordinary visible copy – Page can contain Text directly
- Image renders PNG and JPG – no SVG files (use Svg components instead)
- No DOM elements inside Document – custom components must resolve to react-pdf primitives

### ⚠ Common Mistake

A raw string directly under Page or View triggers a console warning and is omitted. Put ordinary copy in Text; text-capable primitives such as Link are exceptions.

# Styling System

React-PDF styles are JavaScript objects, like React Native – not CSS strings or className props. Tokenize reusable choices; name one-off geometry locally.

```tsx
import { StyleSheet } from '@react-pdf/renderer';
import { colors, spacing, borders } from '../../styles/theme';
const styles = StyleSheet.create({
container: {
flexDirection: 'row',
padding: spacing.lg,
backgroundColor: colors.neutral[50],
borderRadius: borders.radius.md,
},
});
```

## Supported Layout: Flexbox First

Column is the default. Normal flow uses flexbox – no Grid, floats, or inline/block display. Relative and absolute positioning, plus inline style objects, are valid.

| Supported | Not Supported |
| --- | --- |
| flexDirection, alignItems, justifyContent | CSS Grid (grid-template-*) |
| width, height, minWidth, maxWidth | float, clear |
| margin, padding (all sides) | display: inline / block / table |
| position: relative / absolute | box-shadow, text-shadow |
| border (width, color, style, radius) | CSS animations / transitions |
| backgroundColor, opacity, @media queries | calc(), CSS variables, background-image |

# Units

The default unit is points (pt). 1 inch = 72 points. You can also use strings with unit suffixes.

| Unit | Example | Notes |
|---|---|---|
| Points | 16 | Default – no suffix needed |
| Inches | "1in" | 1in = 72pt |
| Millimeters | "25.4mm" | 25.4mm = 1in = 72pt |
| Centimeters | "2.54cm" | 2.54cm = 1in = 72pt |
| Percentage | "50%" | Relative to parent container |

# Colors

Hex, RGB, RGBA, HSL, and named colors all work. Define your palette once in a theme file and reference the constants everywhere.

```tsx
export const colors = {
navy: '#0f2942',       // hex
gold: 'rgb(245, 184, 73)', // rgb
muted: 'hsl(215, 16%, 47%)', // hsl
overlay: 'rgba(15, 41, 66, 0.6)', // rgba with alpha
};

// Reference everywhere – change once, update the whole document
<Text style={{ color: colors.navy }}>Heading</Text>
```

# Font Registration

Three font families are built in: Courier, Helvetica, and Times-Roman, each with bold and italic variants. For anything else, you register fonts explicitly.

```tsx
import { Font } from '@react-pdf/renderer';
import path from 'path';
const dir = path.resolve(__dirname, '../fonts');
Font.register({ family: 'Inter', fonts: [
{ src: path.join(dir, 'Inter-Regular.ttf'), fontWeight: 400 },
{ src: path.join(dir, 'Inter-Bold.ttf'), fontWeight: 700 },
] });
```

### ⚠ Font Limitations

Only TTF and WOFF formats work – OTF files and variable fonts are not supported. Missing styles fail; missing weights resolve to the nearest registered weight.

# Using Registered Fonts

Reference a registered family by name and request a weight or style. Register the variants you intend to use so nearest-weight fallback never changes the design unexpectedly.

| Style | Resolves to | Common use |
|---|---|---|
| fontWeight: 400 | Inter-Regular.ttf | Body copy, captions |
| fontWeight: 700 | Inter-Bold.ttf | Headings, emphasis |
| fontWeight: 600 | Inter-Bold.ttf | Nearest fallback |

## Page Breaking

- wrap={true} on Page – allows content to overflow to next page (default)
- break={true} on View/Text – forces a break when the ancestor Page wraps
- fixed={true} on View – repeats on each subpage produced by its Page
- minPresenceAhead={number} – moves forward when too little can follow on a wrapping Page
- orphans/widows on Text – minimum lines around a split on wrapping pages

> **Tip**
>
> Use Text's render prop for pageNumber and totalPages. View's typed render prop exposes pageNumber and subPageNumber. Example: `render={({ pageNumber }) => pageNumber}`

## A Complete Example

A fixed footer, a forced break before a new chapter, and a card that refuses to split – combined in one tree:

```tsx
<Page wrap>
<View fixed style={footer}>
<Text render={({ pageNumber, totalPages }) =>
`${pageNumber} / ${totalPages}`} />
</View>
<View break>
<Text>Chapter 2 starts on a fresh page</Text>
</View>
<View wrap={false}>
{/* This card stays whole or moves down */}
</View>
</Page>
```

# Putting It Together

These fundamentals – the document/page structure, JavaScript-object styling, flexbox layout, font registration, and page breaking – are the building blocks. Every page in this book uses them.

Internalizing these constraints is the first step toward mastery. Once you stop reaching for web-specific CSS and embrace the react-pdf primitives – no grid, no CSS display: inline/block, no CSS variables – you can leverage the engine's predictable layout behavior to build complex, multi-page documents with confidence.

The next chapter shows how to organize these building blocks into a project architecture that AI agents can work with efficiently. The goal: a structure where AI can edit one page without reading (or breaking) the rest.

| Concept | Key Rule |
| --- | --- |
| Component structure | Document > Page; Page accepts react-pdf primitives directly |
| Styling | JavaScript objects or arrays – no CSS text or className |
| Layout | Flexbox for normal flow – default direction is column |
| Fonts | Register TTF/WOFF explicitly – no synthesized bold/italic |
| Page breaks | wrap, break, fixed, minPresenceAhead props |
| Units | Points (default), inches, mm, cm, percentages |

CHAPTER 03

# Project Architecture for AI Agents

Structure your codebase so AI can edit one page without breaking the rest.

# Why Monoliths Break for AI

The most common react-pdf mistake is putting everything in one file. Your entire 30-page document lives in a single 2,000-line component. You paste it into an AI prompt. The AI edits page 12 and accidentally breaks the styling on page 4.

Big files also strain AI attention – Chapter 5 covers the token math in detail. The structural fix in this chapter is what stops the cross-page breakage even before tokens enter the picture.

# File-Per-Page Pattern

Split every page into its own component file. One file, one page, one concern.

```bash
src/
pages/
01-cover/01-cover.tsx
02-toc/01-toc.tsx
03-introduction/00-title.tsx       # chapter divider
03-introduction/01-introduction.tsx
components/   # Header, Footer, ChapterTitle, CodeBlock, Table, ...
styles/
theme.ts        # design tokens
shared.ts       # shared StyleSheet
manifest.ts       # chapter structure (source of truth)
Document.tsx      # maps the generated registry into one <Document>
build.tsx         # two-pass render to output/react-pdf-ai-builders-guide.pdf
```

## Why This Works

- Page files stay small and focused – often a few dozen lines
- AI can edit one page (e.g. 05-architecture/05-naming-conventions.tsx) without touching the others
- You give AI context: the page file + theme.ts + relevant components = ~3,000 tokens
- Components enforce consistency – every content page uses the same Header and Footer

## Assembly File

The generated registry imports every page. Document.tsx imports that ordered list and maps it into one Document, so the assembly never drifts from the page tree.

```tsx
// Document.tsx (metadata abridged)
import React from 'react';
import { Document } from '@react-pdf/renderer';
import { allPages } from './registry';
const EbookDocument: React.FC = () => (
<Document
title="React-PDF + AI: The Builder's Guide"
author="Landon Miles"
>
{allPages.map((Page, index) => <Page key={index} />)}
</Document>
);
export default EbookDocument;
```

## Build Script

The simplest build script renders the document to a PDF in one call – run it with tsx or ts-node. (This book's own build.tsx adds a second render pass to fill in real TOC page numbers.)

```tsx
// build.tsx
import ReactPDF from '@react-pdf/renderer';
import EbookDocument from './Document';
ReactPDF.render(<EbookDocument />, './output/react-pdf-ai-builders-guide.pdf')
.then(() => console.log('PDF generated: output/react-pdf-ai-builders-guide.pdf'));
```

# Shared Components

Components that appear on multiple pages live in the components/ folder. This is where consistency comes from.

| Component | Purpose | Used By |
|---|---|---|
| Header | Book title + section name at top | Every content page |
| Footer | Page number + branding at bottom | Content pages + TOC |
| ChapterTitle | Full-page chapter opener | Start of each chapter |
| ContentPage | Standard page with header/footer | All content pages |
| CodeBlock | Styled code display | Technical chapters |
| TipBox / WarningBox | Callout boxes | Any page with key info |
| Table | Flexbox-based data table | Comparison/reference pages |
| BulletList | Consistent bullet lists | Throughout |
| Icons (SVG) | Vector icons for visual interest | Headers, callouts, lists |

# Naming Conventions

Chapter files follow a strict pattern: NN-chapter/, with 00-title.tsx for the divider and NN-topic.tsx for content. Cover, TOC, and conclusion are chrome exceptions: each has one 01- file and no divider. Components are PascalCase.tsx. Numeric prefixes keep files sorted and make the target unambiguous. The chapter structure itself lives in src/manifest.ts.

### ⚠ Anti-Patterns

Avoid these structures – they make AI editing harder and more error-prone: monolithic single-file documents, inline styles on every element (use StyleSheet.create), deeply nested component trees that span pages, importing all components everywhere instead of just what's needed.

# The Pattern at a Glance

Chapter paths resolve to a handful of shapes; chrome uses the exception above. Once your AI internalizes the rule, it can name new files correctly – and you can predict where anything lives.

| Pattern | Example | What it is |
|---|---|---|
| NN-chapter/ | 05-architecture/ | One folder per chapter |
| 00-title.tsx | 00-title.tsx | The chapter divider page |
| NN-topic.tsx | 05-naming-conventions.tsx | A single content page |
| PascalCase.tsx | CodeBlock.tsx | A reusable component |
| manifest.ts | src/manifest.ts | The chapter structure |

## How Changes Propagate

When you change a design token in theme.ts, every consumer updates on the next build. Change primary[800], and the heading roles that use it update. Change typography.body.fontSize, and copy using the body preset adjusts. This is centralized decision-making with distributed rendering.

The same applies to shared components. Update the Footer component, and every content page gets the new footer. Fix a bug in CodeBlock, and every code example benefits. You never have to hunt through a monolith file to find every instance of a repeated pattern.

A useful rule of thumb: if a style block or component appears on 3+ pages, extract it into the shared folder. If it only appears once, keep it local to that page file. This keeps the shared layer focused and avoids bloat while still centralizing everything that needs to be consistent.

## One Edit, Many Pages

The system hinges on importing values instead of repeating them. Each heading role references a named color, so a token edit reaches every component that consumes it on the next build:

```ts
// src/styles/theme.ts – reusable design tokens (excerpt)
export const colors = {
primary: { 800: '#121F3D' /* ...9 more steps */ }, // change here ...
accent:  { 500: '#F0A000' /* ...9 more steps */ }, // ...every consumer follows
};
```

| You change... | What updates | Blast radius |
|---|---|---|
| A design token | Every component consuming it | Targeted to book-wide |
| A shared component | Every page using it | Many pages |
| One page file | Only that page | Single page |

CHAPTER 04

# Specifying a Design Language

Define your visual system once. Use it everywhere. Let AI enforce it.

# Why AI Needs Explicit Design Tokens

Without a design system, every AI-generated page is a coin flip. The AI picks fontSize 14 on one page and 16 on the next. It uses #333 for body text here and #444 there. The result looks like five different people designed it.

A design token file solves this. It centralizes reusable visual decisions: colors, font sizes, spacing, and border radii. Include it in every AI prompt so generated pages inherit the same constraints.

# Color Palette

Pick one primary role, one accent role, and a neutral scale. Three to five core roles handle most document design; tonal steps provide useful variation without adding new roles.


This book uses navy primary (#121F3D) with amber/gold accent (#F0A000) and slate neutrals. The navy conveys trust and professionalism. The gold adds warmth and draws the eye to important elements.

**Tip**

Use darker primary shades (700-900): 800-900 for major headings and surfaces, 700 for minor headings. Use the accent for highlights and chapter numbers, and semantic colors for status callouts. Neutrals handle body text, dividers, and secondary information.

# Typography Scale

Define every text size upfront. No magic numbers in page components. Page chrome – this book's 42pt cover title, 32pt chapter titles – comes from a separate fontScale token.

| | | |
|---|---|---|
| `display` | 36pt | Hero/display text |
| `h1` | 26pt | Top-level headings |
| `h2` | 20pt | Section headers |
| `h3` | 16pt | Subsections |
| `h4` | 13pt | Labels, small headers |
| `body` | 11pt | Main reading text |
| `bodySmall` | 9.5pt | Secondary text |
| `caption` | 8.5pt | Footnotes, captions |
| `code` | 9pt | Code blocks |
| `codeSmall` | 8pt | Dense code, labels |

# One Source of Truth

Encode the scale as an object in theme.ts and read every size from it. When you tell your AI agent to build a page, it pulls from these tokens – so headings stay proportional and nothing drifts into a stray 14.5pt.

```tsx
export const typography = {
display: { fontSize: 36, fontFamily: fonts.heading, fontWeight: fontWeight.bold },
h2:      { fontSize: 20, fontFamily: fonts.heading, fontWeight: fontWeight.semibold },
body:    { fontSize: 11, fontFamily: fonts.body, fontWeight: fontWeight.regular },
} as const;
```

## Spacing Scale

Use a mostly 4-point grid with named 1pt and 2pt micro exceptions. Reusable gaps and padding come from it; page geometry has separate named tokens:

```typescript
export const spacing = {
none: 0, micro: 1, xxs: 2,       // zero + micro exceptions
xs: 4, sm: 8, md: 12, lg: 16,    // 4pt core
xl: 24, xxl: 32, xxxl: 48,       // major spacing
};
```

## Theme File

Tokens live in theme.ts. Pages consume them directly or through shared styles/components; reusable design values are not hardcoded.

```typescript
import { colors, fonts, typography, spacing } from '../../styles/theme';
const s = StyleSheet.create({
title: {
...typography.h1,
color: colors.primary[800],
marginBottom: spacing.lg,
},
});
```

### The AI Context Pattern

When prompting AI to create a new page, include theme.ts in the context. The AI will use your defined tokens instead of inventing its own values.

## Border and Radius Tokens

Define border widths and corner radii alongside your other tokens. Without these, AI mixes 4pt rounded corners on one card with 8pt on another – subtle but noticeable.

```typescript
export const borders = {
thin: 0.5,      // Dividers, table cell borders
medium: 1,      // Card borders, table header rules
thick: 2,       // Callout left borders, emphasis
radius: {
xs: 1.5,      // Fine details, compact elements
sm: 3,        // Callout boxes, badges, tags
md: 6,        // Cards, code blocks, tables
lg: 10,       // Feature cards, hero elements
},
};
```

## Semantic Color Tokens

Beyond your core palette, define semantic colors for meaning: success (green), warning (amber), error (red), info (blue). Include light variants for callout backgrounds.

```typescript
success: '#2D8B4E',  successLight: '#F0F9F4',
warning: '#D98E00',  warningLight: '#FEF8E6',
error:   '#C43333',  errorLight:   '#FEF3F3',
info:    '#2E6BB5',  infoLight:    '#EDF1F8',
```

### The Complete Token Set

Start with four core categories: colors (5-8 base values), typography (7-9 sizes), spacing (7-9 values), and borders (3 widths + 4 radii), plus semantic colors for callouts. Then promote recurring magic numbers into new tokens – this book's theme.ts grew to 15 categories.

# Design Token Checklist

| Category | Count | Example |
| --- | --- | --- |
| Colors (primary + accent) | 20 shades (10 each) | #121F3D navy, #F0A000 gold |
| Neutrals + semantic | 5-10 + 4 pairs | Gray scale + success/warning/error/info |
| Typography | 10 presets | 8pt codeSmall to 36pt display |
| Spacing | 9 positive + zero | 1pt micro to 48pt xxxl |
| Borders | 3 widths + 4 radii | 0.5pt thin to 10pt lg radius |

# Auditing Your Tokens

Run your theme.ts through this starter list before you build a single page – then let it grow. This book's theme reached 15 categories, from font weights to icon sizes:

- Reusable design values live in theme.ts; unique geometry uses named local constants instead of scattered literals.
- Each scale has an intentional progression – a mostly 4pt spacing grid with micro exceptions and a curated type hierarchy.
- Semantic colors map to meaning – success, warning, error, info – not to a specific shade you might want to swap later.
- Names describe role, not appearance – primary[800] over navyDark, so a rebrand is a centralized palette change.

CHAPTER 05

# Tokenization and Context Windows

How LLMs read your code – and how to make every token count.

## What Tokens Are

LLMs don't read characters or words. They read tokens – subword units that the model was trained to recognize. In English text, one token is roughly 4 characters or 0.75 words. Code tokenizes differently because of syntax characters.

This matters for react-pdf because your page components are code. The angle brackets, prop names, style objects, and JSX structure all consume tokens. A simple page component with a few Text elements and a StyleSheet might be 200-400 tokens. A complex page with tables and code blocks could be 600-1,000 tokens.

## Context Window Math

Every AI model has a context window – the total amount of text it can process in a single conversation turn. Specific models change frequently, but the categories remain stable:

| Category | Context Window | Effective Working Memory |
|---|---|---|
| Standard models | 32K-128K tokens | ~4,000-12,000 tokens (sweet spot) |
| Large-context models | 128K-200K+ tokens | ~10,000-20,000 tokens (sweet spot) |
| Extended-context models | 500K-1M+ tokens | ~20,000-40,000 tokens (sweet spot) |

### ⚠ The 'Lost in the Middle' Problem

Research shows LLMs pay most attention to the beginning and end of their context. Content in the middle gets less attention. A 15,000-token monolithic PDF file means your AI is likely ignoring the pages in the middle – exactly where bugs hide.

# Token Cost of React-PDF Code

JSX is token-expensive compared to plain text. Here's what typical react-pdf structures cost:

| Structure | Approximate Tokens |
| --- | --- |
| A StyleSheet.create() with 10 style objects | 200-350 |
| A simple page (title, 3 paragraphs) | 150-250 |
| A complex page (table, code block, callouts) | 400-800 |
| A shared component (Header or Footer) | 400-500 |
| A theme.ts with full design tokens | 1,500-1,700 |
| A 30-page monolith file | 8,000-15,000 |

# Practical Strategy

When you ask AI to edit a page, it needs context. Here's the optimal context budget:

- theme.ts (design tokens): ~1,650 tokens
- Relevant shared components: ~300-500 tokens
- The page file being edited: ~200-600 tokens
- Your instructions: ~100-300 tokens
- Total: 2,200-3,000 tokens of context – well within any model's sweet spot

Compare that to the monolith approach: 8,000-15,000 tokens dumped into context, with your edit instructions buried somewhere inside. The file-per-page architecture isn't just about code organization. It's about giving AI the right amount of focused context.

## Reducing Token Waste

- Extract repeated styles into shared.ts – define once, reference everywhere
- Use short but descriptive variable names – "s" for local styles is fine
- Don't add comments that restate the obvious – "// Render the title" above a title render
- Keep page components focused – if a section could be its own page, make it one
- Use docs/ for concise project guidance; keep long-form research in reference/

## Prompt Sizing in Practice

Here's what an efficient AI prompt looks like. Notice how small and focused the context is:

```text
Context files (paste these):
1. theme.ts (your design tokens)
2. ContentPage.tsx (the wrapper component)
3. 07-tokenization/04-reducing-waste.tsx (target page)

Instruction:
"Add a new section called 'Prompt Sizing' after
the BulletList. Use a SectionHeading for the
heading and styles.body for the paragraph. Include
a CodeBlock showing an example prompt."
```

Total context: roughly 2,650 tokens. The AI has everything it needs to produce code that matches your design system, uses the correct component wrapper, and fits the existing page structure. No guesswork required.

### Project Docs vs. Research

Keep concise project guidance – design tokens, component API, conventions – in docs/. Reserve reference/ for long-form research. Point AI to the smallest relevant docs instead of loading all source files.

### Token Budget Template

For each AI edit session, aim for this budget: theme (1,650) + components (350) + target page (450) + instructions (200) = 2,650 tokens. In a 128K context window, that leaves about 98% available for reasoning and output.

## Measure Your Own Files

Estimates lie. Real numbers don't. Run this in your project root for a per-file token estimate (4 chars ≈ 1 token):

```bash
# Approximate token count per .tsx file, largest first
find src -name '*.tsx' -exec wc -c {} \; \
| awk '{ printf "%6d tokens  %s\n", $1/4, $2 }' \
| sort -rn | head -20
```

Page or feature files over 1,500 tokens are worth reviewing for a split. Cohesive source-of-truth files such as theme.ts can stay larger when dividing them would make the system harder to follow. Then sum the files an AI actually needs for one edit (theme + components + target page + instructions). A total under 4,000 is compact even for a 32K model and uses just over 3% of a 128K context window.

CHAPTER 06

# Avoiding AI Slop

What makes output look cheap – and the specific fixes that make it premium.

## What AI Slop Looks Like

You've seen it. The AI generates a PDF and it looks... generic. Default font. Random spacing. Every page feels like a different document. Here's what specifically goes wrong:

- Default Helvetica everywhere – no typographic personality
- Inconsistent font sizes – 14pt here, 16pt there, no clear hierarchy
- Cramped margins – content pushed to the edges with no breathing room
- Random colors – #333, #666, #007bff borrowed from different templates
- No visual hierarchy – body text and headings nearly the same size
- Walls of text – no callout boxes, no bullet lists, no visual breaks
- Lorem ipsum remnants – placeholder text the AI forgot to replace

## Root Causes

AI slop isn't the AI's fault. It's a prompt and structure problem.

- Lazy prompts: "Make me a PDF" gives the AI nothing to work with
- No design system: without tokens, the AI improvises – badly
- No visual QA: you accept the first output without reviewing the render
- Monolith structure: the AI can't focus on one page at a time
- No iteration: premium output takes 2-3 passes, not one

### ⚠ The First Draft Rule

AI's first attempt is a rough draft. Always. It gives you the structure and general direction, but the spacing, sizing, and visual balance need refinement. If you ship the first draft, you ship slop.

# Slop vs. Premium – Side by Side

## Heading Styles

**Slop**

fontSize: 16, centered, default font, no color, no spacing above or below. Blends into body text.

**Premium**

Uses typography.h2 (20pt), primary color, 16pt marginBottom, 24pt marginTop. Clear hierarchy break.

## Page Margins

**Slop**

padding: 20 on all sides. Content cramped against edges. Feels claustrophobic.

**Premium**

54pt horizontal, 60pt vertical margins. Content has room to breathe. Professional whitespace.

## Color Palette

**Slop**

Pure black (#000) ink with a saturated default-blue link. Harsh contrast, screen-not-print feel.

**Premium**

Deep navy ink, never pure black, with one restrained gold accent – a few tokens, used consistently everywhere.

### The Pattern

Slop comes from accepting library defaults; premium comes from a small set of deliberate tokens applied without exception. The fix is rarely more design – it is fewer decisions, made once, and reused on every page.

## Iteration Workflow

Premium output requires a loop, not a single prompt:

- Pass 1: Generate structure and content – get the text right
- Pass 2: Apply design system – enforce tokens, fix spacing
- Pass 3: Visual QA – render to PNG, review, fix alignment and balance
- Optional Pass 4: Polish – add callouts, refine typography, check page breaks

### The Prompt That Fixes Slop

Instead of "make me a page about X", try: "Create a content page using ContentPage wrapper. Use typography from theme.ts: h2 for section titles, body for paragraphs. Include a TipBox callout and a BulletList. Match the spacing and style of 05-architecture/01-architecture.tsx."

## Knowing When to Stop

The loop has a floor. Past a certain point, more passes add churn, not quality – and over-polishing reintroduces its own kind of slop. Stop when:

- The page renders cleanly with no orphaned headings, split callouts, or stray bullet dots
- Every reusable color, font, and spacing value traces back to a named token
- The layout reads at a glance – clear hierarchy, comfortable density, balanced whitespace
- A fresh read surfaces only nitpicks you would not pay to fix

### Diminishing Returns

If your AI keeps "improving" a page that already passes QA, you are spending tokens to move pixels. Lock the page, commit it, and move the loop to the next one.

# Prompt Templates That Prevent Slop

The prompts that produce premium react-pdf output share one trait: they front-load constraints so the AI can't improvise. The template below is copy-paste ready – fill the bracketed slots and run it. Six fuller templates ship in the repo's templates/ directory.

## New Page

```
Build a new ContentPage for [TOPIC].

Reference: src/pages/[EXISTING_PAGE].tsx for structure.
Design system: src/styles/theme.ts (tokenize reusable choices).
Components: SectionHeading, BulletList, CodeBlock, TipBox, Table.

Requirements:
- Use ContentPage wrapper with sectionTitle="[SECTION]"
- Import styles from '../../styles/shared'
- Every text run must resolve to a registered family and weight
- Use shared typography styles; pair fontFamily + fontWeight when defining either
- wrap={false} on elements that must not split across pages
- No Helvetica, no emoji, no hardcoded colors
```

### The Negative Constraint Pattern

Three or four negative constraints ("no Helvetica," "no emoji," "no hardcoded colors") consistently produce cleaner output than twice as many positive instructions. LLMs predict the most likely next token – negative constraints override bad defaults and close escape hatches the AI would otherwise use.

# Copy-Paste Test

Open a fresh AI session with no prior context. Paste only the template plus the files it references. If the generated page matches your existing pages, the prompt is good enough; if not, the gap reveals which constraint is missing.

CHAPTER 07

# Design Challenges and Solutions

What works, what breaks, and the workarounds that actually hold up.

## What Works Well

React-PDF will frustrate you if you fight it. Don't. These patterns work cleanly – lean into them and you'll build faster than you expect:

- Single-column layouts with clear heading hierarchy
- Two-column layouts using flexDirection: "row" with fixed widths
- Full-width colored banners and pull quotes with accent borders
- Consistent card patterns: bordered View with padding and borderRadius
- SVG decorative elements (lines, circles, icons)
- Bullet lists and tables built with flexbox rows
- Page wrapper inserts headers/footers; fixed={true} repeats them on wrapped pages

## What to Avoid

The flip side: a handful of patterns that feel reasonable in a browser but quietly break in PDF. Reach for them and you'll spend your afternoon debugging layout instead of shipping pages:

- Percentage heights in an unsized parent – they resolve against nothing and collapse
- Floated or grid layouts – use flexbox rows and columns for normal flow
- position: "fixed" – no such style value; repetition comes from the fixed={true} prop
- Deeply nested Views chasing pixel-perfect alignment – flatten and let flex do the work

### ⚠ Watch out

When a layout misbehaves, an unsupported CSS property is a common cause. Check the supported-style list before you start bisecting, then inspect wrapping, content height, and flex constraints.

# What Doesn't Work (and Fixes)

## ✕ CSS Grid Layouts

Not supported. Use nested flexbox with explicit widths instead:

```tsx
// Three-column "grid" using flexbox
<View style={{ flexDirection: 'row' }}>
{['Column 1', 'Column 2', 'Column 3'].map((label) => (
<View key={label} style={{ width: '33%', padding: spacing.sm }}>
<Text>{label}</Text>
</View>
))}
</View>
```

## ✕ Drop Shadows

box-shadow is not supported. Use a subtle border or an offset nested View:

```tsx
// Fake shadow via darker bottom/right borders
<View style={{
borderWidth: borders.medium,
borderColor: colors.neutral[200],
borderRadius: borders.radius.md,
borderBottomWidth: borders.thick,
borderRightWidth: borders.thick,
borderBottomColor: colors.neutral[300],
borderRightColor: colors.neutral[300],
padding: spacing.lg,
backgroundColor: colors.white,
}}>
<Text>Card with faux shadow</Text>
</View>
```

## ✕ Text Wrapping Around Images

Float is unsupported. Use a row with named one-off geometry:

```tsx
const mediaSize = { width: 120, height: 90 };
<View style={{ flexDirection: 'row', gap: spacing.lg }}>
<Image src="photo.png" style={mediaSize} />
<View style={{ flex: 1 }}>
<Text>Text flows in a column beside
the image, not wrapped around it.
</Text>
</View>
</View>
```

## ✕ Gradient Backgrounds

CSS gradients need SVG. Name the layer height once:

```tsx
const GRADIENT_HEIGHT = 100;
<View style={{ position: 'relative', height: GRADIENT_HEIGHT }}>
<Svg style={{ position: 'absolute', top: 0, left: 0, right: 0, bottom: 0 }}
viewBox={`0 0 ${page.contentWidth} ${GRADIENT_HEIGHT}`}>
<Defs>
<LinearGradient id="bg" x1="0" y1="0"
x2="1" y2="0">
<Stop offset="0%" stopColor={colors.primary[700]} />
<Stop offset="100%" stopColor={colors.primary[500]} />
</LinearGradient>
</Defs>
<Rect width={page.contentWidth} height={GRADIENT_HEIGHT}
fill="url(#bg)" />
</Svg>
<Text style={{ color: colors.white, padding: spacing.lg }}>
Content over gradient
</Text>
</View>
```

# Recipe: Professional Table

Tables in react-pdf are built with flexbox rows, not HTML table elements. The key ingredients: a dark header row with white semibold text, alternating row backgrounds for scan-ability, consistent padding, and subtle bottom borders. Pass percentage columnWidths like '30%' and keep column counts low – this book tops out at four before LETTER or A4 gets cramped. The kit ships this recipe as the Table component (src/components/Table.tsx).

```tsx
const DataTable = ({ headers, rows, columnWidths }) => (
<View style={styles.tableContainer}>
{/* Header row */}
<View wrap={false} style={styles.tableHeader}>
{headers.map((h, i) => (
<Text key={i} style={[styles.tableHeaderText,
{ width: columnWidths[i] }]}>{h}</Text>
))}
</View>
{/* Data rows with alternating colors */}
{rows.map((row, ri) => (
<View key={ri} wrap={false} style={ri % 2 === 1
? styles.tableRowAlt : styles.tableRow}>
{row.map((cell, ci) => (
<Text key={ci} style={[styles.tableCell,
{ width: columnWidths[ci] }]}>{cell}</Text>
))}
</View>
))}
</View>
);
```

Every workaround in this chapter follows the same pattern: the CSS property you want doesn't exist, so you build it from View, Text, and flexbox. That's not a limitation – it's the entire design philosophy. Once you stop reaching for web-centric CSS and start thinking in react-pdf primitives, layouts get faster to build, not slower.

CHAPTER 08

# Icons over Emojis

SVG icons render sharp, match your brand, and work offline.

# Emoji Trade-offs in PDFs

Emojis look harmless. They're quick to type and AI agents love inserting them. But in react-pdf, they introduce real problems:

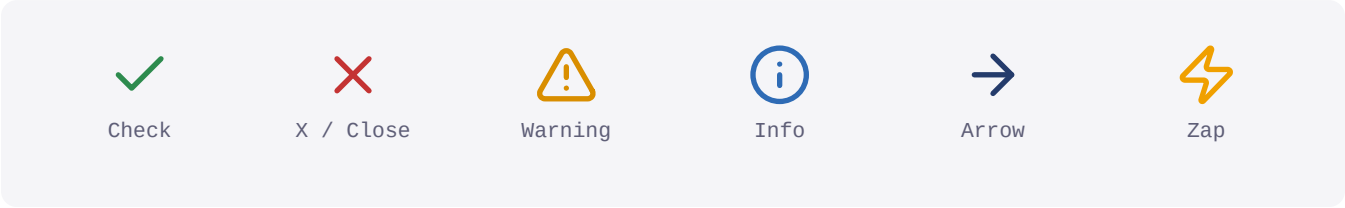
| Issue | Impact |
| --- | --- |
| Needs an emoji image source | Remote sources add network risk; vendored sources work offline |
| Raster images at fixed resolution | Blurry when zoomed – PDFs are vector-first |
| Source-dependent artwork | A fixed source renders consistently across platforms |
| Color baked into the artwork | Cannot be recolored with document tokens |
| Informal appearance | Undercuts professional documents |
| Remote CDN outage | Breaks builds only when the selected source is remote |

### ⚠ Remote Source Risk

Font.registerEmojiSource() can use a remote set such as Twemoji or a vendored local image set. A remote source makes builds depend on the network and its host; vendor the assets when offline and reproducible rendering matter.

# SVG Icon Advantage

React-PDF has extensive SVG support. SVG icons are vector graphics – they scale cleanly at any zoom level, match any color, and render consistently across PDF viewers with no network or CDN dependency at render time.




These icons come from Lucide via the react-icons package (ISC license). A small adapter rewrites react-icons' browser SVG output as @react-pdf/renderer Svg/Path nodes. Size and color stay under your control; stroke weight follows the source icon unless you expose it as another adapter prop.

# Why Not Emojis?

| Property | SVG icon | Emoji glyph |
|---|---|---|
| Color | Any token you want | Fixed by the artwork |
| Rendering | Consistent vector path | Depends on chosen source |
| Stroke weight | Set by source icon | None |
| Print fidelity | Crisp vectors | Often missing |

## react-icons Adapter

react-icons returns plain HTML <svg>/<path> elements, which react-pdf can't render. The adapter walks the icon's React tree once and rebuilds it with react-pdf primitives, normalizing currentColor and string-typed numerics along the way.

```tsx
import { Svg, Path, Circle, Line } from '@react-pdf/renderer';
import type { IconType } from 'react-icons';
import { iconSize } from '../styles/theme';
const TAG_MAP = { svg: Svg, path: Path, circle: Circle, line: Line /* ...etc */ };

const Icon = ({ icon, size = iconSize.lg, color }) => {
const { attr, children } = icon({}).props;
const svgProps = coerceProps(attr, color);
return (
<Svg width={size} height={size} {...svgProps}>
{convertChildren(children, color, svgProps)}
</Svg>
);
};
```

## Using Icons in Context

- Vector-sharp at any zoom level
- Matches your brand color palette
- Zero network dependencies at render time – no CDN, no internet

### When Emojis Are Acceptable

Internal team documents, quick prototypes, or documents where your brand literally uses emoji as part of its identity. For anything client-facing or paid – use SVG icons.

# Skip the Path Data – Use react-icons

The book's source ships an Icon adapter for compatible react-icons IconType values built from its mapped SVG primitives. The package offers nearly 50,000 vetted icons (Lucide, Heroicons, Feather, Tabler, Font Awesome) – no path-string copying or SVG hand-tuning for compatible icons.

```tsx
import { LuCheck, LuTriangleAlert } from 'react-icons/lu';
import Icon from './components/Icon';
import { colors, iconSize } from './styles/theme';
<Icon icon={LuCheck} size={iconSize.md} color={colors.success} />
<Icon icon={LuTriangleAlert} size={iconSize.lg} color={colors.warning} />
```

Re-export the icons your document actually uses from a single Icons.tsx so prompts stay short. Keep that re-export file under ~12 icons; ad-hoc **react-icons/lu** imports are fine for one-off uses.

### ⚠ Path Data Quality

Not every icon library renders cleanly. Stick to stroke-based sets (Lucide, Feather, Heroicons-outline). Avoid icons that rely on filters or masks – react-pdf supports neither, and the adapter drops unmapped tags silently.

# Which Sets Render Cleanly

| Import prefix | Library | Style | Use it? |
|---|---|---|---|
| react-icons/lu | Lucide | Stroke outline | Yes – first choice |
| react-icons/fi | Feather | Stroke outline | Yes – minimal set |
| react-icons/hi2 | Heroicons v2 | Outline + solid | Yes – use outline |

## Sizing Cheat Sheet

Match icons to the text beside them – oversized looks amateur, undersized disappears.

| Context | Icon Size | Text Size |
|---|---|---|
| Inline with body text | 12-14pt | 11pt body |
| List markers | 6pt dot / 10-12pt icon | 9.5-11pt |
| Section headers | 16pt | 20pt h2 |
| Feature showcases | 24pt | Display or standalone |
| Callout box labels | 13pt | 13pt label |

## Emoji vs Icon, Side by Side

| Property | Emoji | SVG Icon |
|---|---|---|
| Rendering | Source-dependent | Consistent vector path |
| Sizing | Fixed to font size | Any size, precise control |
| Color | Cannot be changed | Any color from your palette |
| Print quality | May rasterize poorly | Vector – infinite resolution |
| AI consistency | Depends on source setup | Deterministic output |

CHAPTER 09

# AI Visual Analysis

Export to PNG. Let AI see what the reader sees. Fix what doesn't look right.

# Why PNGs Beat Raw PDFs for AI Review

AI tools preprocess PDFs differently, often combining extracted text with page images. For visual QA, control that conversion: export PNGs at a chosen DPI so the model reviews a consistent rasterization of the same PDF layout your readers receive.

# Visual QA Workflow

1 Generate PDF → 2 Export to PNG → 3 AI Reviews PNG → 4 Fix & Repeat

# PDF to PNG Conversion

The most reliable tool is pdftoppm from the poppler-utils package. It's fast, handles edge cases well, and produces clean output. DPI trades image quality against file size – 200 DPI is the sweet spot. Sizes below are measured from this book's text-heavy pages.

| DPI | Use Case | File Size |
|---|---|---|
| 72 | Quick preview, thumbnail | ~15-105KB per page |
| 150 | Standard review, good detail | ~35-265KB per page |
| 200 | Detailed AI analysis (recommended) | ~50-320KB per page |
| 300 | Print-quality verification | ~80-440KB per page |

## Export Script

Wire the export into package.json: "pnpm build" generates the PDF, "pnpm export" rasterizes it to PNGs, and "pnpm pipeline" chains both. The script below is simplified – the full version also checks for a missing PDF or pdftoppm.

```bash
#!/bin/bash
# scripts/export-pages.sh (simplified)
set -e
PDF_FILE="output/react-pdf-ai-builders-guide.pdf"
PNG_DIR="output/pages"
DPI="${1:-200}"  # DPI argument, default 200
mkdir -p "$PNG_DIR"
rm -f "$PNG_DIR"/page-*.png  # clear stale pages
pdftoppm -png -r "$DPI" "$PDF_FILE" "$PNG_DIR/page"
echo "Exported to $PNG_DIR/"
ls -lh "$PNG_DIR"/page-*.png
```

## What AI Can Spot

Be specific in your prompts: "Check if the heading alignment is consistent across pages 3-7" works better than "review this PDF." And review in small batches – assistants cap images per conversation (the Claude API allows 100), so open only the pages you changed. Here's what vision models reliably detect:

- ✓ Alignment inconsistencies between elements
- ✓ Spacing irregularities (uneven margins, padding gaps)
- ✓ Font size mismatches across similar elements
- ✓ Color contrast issues (text hard to read on background)
- ✓ Layout balance problems (too heavy on one side)
- ✓ Missing or broken elements
- ✓ Orphaned lines at page breaks
- ✓ Overall visual hierarchy assessment

## What AI Struggles With

- ✕ Exact pixel measurements (estimates within ~5-10%)
- ✕ Precise color matching (close, not pixel-perfect)
- ✕ Specific font identification
- ✕ Reading small text at low DPI
- ✕ Distinguishing very similar shades

Here's what matters: AI catches relative problems, not absolute ones. It'll tell you "this heading looks different from that heading" but not "this margin is exactly 24px." Use it for what it's good at – alignment, spacing rhythm, visual hierarchy, broken elements – and verify exact values in your code.

## Three Review Prompts That Work

Chapter 6 covered prompts that **generate** pages. These are different – they ask AI to **review** a page that already exists. Single-axis prompts get you specifics:

```text
SPACING: "Look at this PDF page. Are the vertical gaps
between sections consistent? Are any two adjacent
elements too close together or too far apart?"

HIERARCHY: "This page should have 4 levels of text:
page title (largest), section headings, body text,
and captions (smallest). Can you clearly identify all
4 levels? Are any two levels too similar in size?"

CONSISTENCY: "Here are pages 3 and 4 of the same
document. Do the headers look identical? Are the
margins the same? Do the callout boxes use the same
styling? Flag any inconsistencies between the pages."
```

CHAPTER 10

# Premium Deliverables & Recipes

The checklist that separates a free PDF from a $50 one.

# Premium Checklist

Every item below is concrete and verifiable. A PDF that hits all of these looks intentionally designed – not generated.

## Typography

- ✓ Custom font registered (not default Helvetica/Arial)
- ✓ 3+ distinct text sizes used with clear hierarchy
- ✓ Consistent heading styles across all pages
- ✓ Body text at readable size (10-12pt) with 1.5-1.6 line height
- ✓ Code blocks in monospace font with background color

## Color & Visual

- ✓ Palette limited to 3-5 core color roles plus intentional tonal shades
- ✓ Primary color for headings and emphasis areas
- ✓ Accent color for highlights, callout borders, interactive elements
- ✓ Neutral scale for body text, borders, and backgrounds
- ✓ Semantic colors carry meaning; decorative accents stay within the brand palette

## Production & Polish

- ✓ Custom fonts embedded; standard PDF base fonts used intentionally
- ✓ File size kept lean – images compressed, no unused assets
- ✓ Selectable, copyable text (not rasterized into images)
- ✓ Final PDF proofed at 100% zoom and at print scale

## Layout & Spacing

- ✓ Generous page margins (50-70pt on all sides)
- ✓ Consistent spacing scale used throughout (4pt base with micro-adjustments)
- ✓ Whitespace used intentionally – pages don't feel cramped
- ✓ No orphaned single lines at page breaks
- ✓ Visual breaks between major sections

## Structure & Navigation

- ✓ Cover page with strong visual identity
- ✓ Table of contents with chapter listing
- ✓ Page numbers on every content page
- ✓ Headers with section context
- ✓ Footer with branding
- ✓ Chapter title pages with distinct design

## Content Components

- ✓ Callout boxes for tips, warnings, and key information
- ✓ Styled code blocks with language labels
- ✓ Professional tables with header rows and alternating colors
- ✓ SVG icons (not emojis) for visual elements
- ✓ Bullet lists with consistent formatting
- ✓ Document metadata set (title, author, subject)

## Quality Tests

- The squint test: step back, squint at the page. Can you see clear visual hierarchy? If everything blurs into one gray block, you need more contrast.
- The scroll test: flip through all pages quickly. Do they feel like one cohesive document? Or do some pages look like they belong to a different PDF?
- The "would I share this?" test: if someone you respect saw this PDF with your name on it, would you be proud? If you hesitate, it needs more work.
- The comparison test: open a professionally designed PDF from a publisher. Hold yours next to it. Where does yours fall short? Fix those specific things.

## From Checklist to Habit

The first time through this checklist takes effort. By the third document, it becomes muscle memory. You'll register custom fonts automatically, reach for your design tokens without thinking, and notice spacing inconsistencies at a glance. The goal isn't to check boxes forever – it's to internalize what premium looks like so your first drafts start closer to the finish line.

### The Design System Payoff

If you built your theme.ts correctly and pages consume it directly or through shared styles and components, most of this checklist is satisfied automatically. The design system isn't extra work – it's the shortcut that makes every page look premium by default.

## Free vs. Premium at a Glance

| Signal | Free / Generic | Premium / Polished |
|---|---|---|
| Fonts | Default Helvetica | Custom registered typeface |
| Colors | Random hex values | Systematic palette with intent |
| Spacing | Inconsistent margins | 4pt base with micro tokens |
| Components | Plain text + emoji | Styled boxes with SVG icons |
| Hierarchy | Everything same size | Clear heading scale with accents |

## Recipes & Templates

These recipes turn the checklist into code. They use Chapter 4's tokens and Chapter 3's shared components; the invoice also reuses Chapter 7's Table. Review those chapters or clone the source if you're starting here.

Three recipes follow: an invoice component (the most-requested PDF use case), a data-driven page generator, and a layout patterns cheat sheet. Each is a self-contained pattern you can copy into your project and customize.

# Recipe: Invoice Component

This invoice component takes a typed data object and renders a header, line items table, and totals. Swap the InvoiceData interface for any typed shape (receipts, purchase orders, quotes) and the structure stays the same.

```tsx
interface InvoiceData {
number: string; due: string;
from: { name: string; address: string };
items: { desc: string; qty: number; price: number }[];
tax?: number;
}

const Invoice = ({ data }: { data: InvoiceData }) => {
const subtotal = data.items.reduce((s, i) => s + i.qty * i.price, 0);
const tax = subtotal * (data.tax ?? 0);
return (
<Page size="LETTER" style={styles.page}>
<View style={{ flexDirection: 'row', marginBottom: spacing.xl }}>
<View style={{ width: '50%' }}>
<Text style={styles.h3}>{data.from.name}</Text>
<Text style={styles.bodySmall}>{data.from.address}</Text>
</View>
<View style={{ width: '50%', alignItems: 'flex-end' }}>
<Text style={styles.h2Text}>INVOICE #{data.number}</Text>
<Text style={styles.bodySmall}>Due: {data.due}</Text>
</View>
</View>
<Table headers={['Description','Qty','Price','Total']}
rows={data.items.map(i => [i.desc, String(i.qty),
'$' + i.price.toFixed(2), '$' + (i.qty*i.price).toFixed(2)])}
columnWidths={['50%','15%','15%','20%']} />
<View style={{ alignItems: 'flex-end', marginTop: spacing.sm }}>
<Text style={styles.body}>Subtotal: ${subtotal.toFixed(2)}</Text>
<Text style={styles.h3}>Total: ${(subtotal+tax).toFixed(2)}</Text>
</View>
</Page>
);
};
```

# Recipe: Data-Driven Pages

Most useful PDFs are generated from data, not static content. The pattern: pass a typed array, .map() to render one page per item, use conditional rendering to control which sections appear. Each page becomes a pure function of its data – change the input and the PDF rebuilds.

```tsx
interface Section {
title: string;
body: string;
metrics?: { label: string; value: string }[];
}

const Report = ({ sections }: { sections: Section[] }) => (
<Document>
{sections.map((section, i) => (
<ContentPage key={i} sectionTitle={section.title}>
<SectionHeading>{section.title}</SectionHeading>
<Text style={styles.body}>{section.body}</Text>

{section.metrics && (
<View style={{ flexDirection: 'row', gap: spacing.md }}>
{section.metrics.map((m, j) => (
<View key={j} wrap={false} style={{
flex: 1,
backgroundColor: colors.neutral[50],
padding: spacing.md,
borderRadius: borders.radius.md,
}}>
<Text style={styles.bodySmall}>{m.label}</Text>
<Text style={styles.h3}>{m.value}</Text>
</View>
))}
</View>
)}
</ContentPage>
))}
</Document>
);
```

# Layout Patterns Cheat Sheet

Four layouts you'll reach for repeatedly. Each is a self-contained pattern you can drop into any ContentPage.

## Two-Column Layout

```tsx
<View style={{ flexDirection: 'row', gap: spacing.lg }}>
<View style={{ flex: 1 }}>
<Text style={styles.body}>Left column</Text>
</View>
<View style={{ flex: 1 }}>
<Text style={styles.body}>Right column</Text>
</View>
</View>
```

## Sidebar + Content

```tsx
<View style={{ flexDirection: 'row', gap: spacing.lg }}>
<View style={{ width: '28%',
backgroundColor: colors.neutral[50],
padding: spacing.md,
borderRadius: borders.radius.md }}>
<Text style={styles.h4}>Sidebar</Text>
</View>
<View style={{ flex: 1 }}>
<Text style={styles.body}>Main content area</Text>
</View>
</View>
```

## Card Grid (2x2)

```tsx
<View style={{ flexDirection: 'row', flexWrap: 'wrap',
gap: spacing.md }}>
{items.map((item, i) => (
<View key={i} wrap={false} style={{
width: '48%', padding: spacing.md,
borderWidth: borders.medium,
borderColor: colors.neutral[200],
borderRadius: borders.radius.md,
}}>
<Text style={styles.h4}>{item.title}</Text>
<Text style={styles.bodySmall}>{item.desc}</Text>
</View>
))}
</View>
```

## Metric / KPI Row

```tsx
<View style={{ flexDirection: 'row', gap: spacing.md }}>
{metrics.map((m, i) => (
<View key={i} style={{ flex: 1, alignItems: 'center',
padding: spacing.md,
backgroundColor: colors.neutral[50],
borderRadius: borders.radius.md }}>
<Text style={styles.bodySmall}>{m.label}</Text>
<Text style={styles.h2Text}>{m.value}</Text>
</View>
))}
</View>
```

## Putting It All Together

- Use Chapter 3's file-per-page structure to keep recipes composable
- Use Chapter 4's tokens for reusable colors and spacing in recipe code
- Apply Chapter 2's wrap={false} patterns to keep recipe cards from splitting across pages
- Clone the source – every pattern here follows conventions used throughout the book
- Explore the react-pdf GitHub repo for additional component examples and API updates

The invoice and report recipes follow typed data in, styled PDF out: interfaces define the contract, components handle layout, and tokens enforce consistency. The layout recipes instead demonstrate reusable composition without a data model. Start with the pattern closest to your use case and build from there.

## The Shape of a Data-Driven Recipe

Strip away the specifics and each data-driven recipe reduces to the same three lines. Hand your AI this skeleton and the recipe page it should follow – it will fill in the interface and layout against your real data.

```tsx
interface ReportData { /* your typed contract */ }

const Report = ({ data }: { data: ReportData }) =>
<Document><Page style={styles.page}>{/* tokens + components */}</Page></Document>;
```

### One Pattern, Many Documents

Invoices, reports, certificates, dashboards – they all share this skeleton. Once your AI internalizes the typed-data-in, styled-PDF-out shape, generating a new document type is mostly a matter of swapping the interface and picking which components to compose.

CHAPTER 11

# Troubleshooting & Common Errors

The errors you'll hit, why they happen, and how to fix them fast.

# Raw Strings Outside a Text Component

### ⚠ Console Warning

"Invalid '...' string child outside <Text> component"

This is the most common react-pdf mistake. In this version, a raw string directly under Page or View triggers a warning and is omitted. Put ordinary copy in Text; text-capable primitives such as Link are exceptions.

```tsx
// Broken – raw string inside View
<View>
This raw string is invalid
</View>

// Fixed – wrap in Text
<View>
<Text style={styles.body}>This renders correctly</Text>
</View>
```

# Fonts Not Loading

### ⚠ Symptom

The build reports an unregistered family or unreadable font file, or text uses a different weight than expected.

An unknown family or bad file path fails the build. For an unavailable weight, react-pdf selects the nearest registered weight. Helvetica is the default only when neither Text nor an ancestor sets fontFamily; this project's ContentPage inherits Inter from its Page style.

# Content Overflows the Page

> ⚠ **Symptom**
>
> Content runs off the bottom of the page, disappearing below the margin. Footer may overlap with body text.

This project renders one non-wrapping ContentPage per source file. If one overflows, shorten or split that file. In a wrapping document, a wrap=false parent or fixed content height can block page breaks.

```tsx
// On a wrapping Page, this child cannot break
<View wrap={false} style={{ minHeight: 800 }}>
{/* Content can't split – overflows the page */}
</View>
// Fix there: allow the parent to wrap, lock each item
<View> {/* wrap={true} is default */}
{items.map((item, i) => (
<View key={i} wrap={false}> {/* Each item stays together */}
<Text style={styles.body}>{item}</Text>
</View>
))}
</View>
```

# Other Culprits to Check

| Property | Why it overflows | Fix |
|---|---|---|
| height / minHeight | Forces a content box taller than the page can hold. | Remove it from content containers. |
| position: absolute | Escapes flow, so react-pdf cannot break it. | Use flex layout for body content. |

# Orphaned Headings

### ⚠ Symptom

A section heading appears at the very bottom of a page with no content following it. The body text starts on the next page.

On a wrapping Page, minPresenceAhead moves the heading forward when too little content can follow it. This project's ContentPage is non-wrapping, so prevent orphans by shortening or splitting the source page during visual QA.

```tsx
{/* Use inside a wrapping Page */}
<View wrap={false} minPresenceAhead={40}>
<Text style={styles.h2Text}>Section Title</Text>
</View>
```

# Split Callout Boxes

### ⚠ Symptom

A colored callout box splits across a page break. The top half shows the colored background, the bottom appears without – looking broken.

Callout boxes should always stay on a single page. The TipBox, WarningBox, and InfoBox components include wrap=false internally. For custom callouts, add wrap=false to the outer View.

# Styles Not Applying

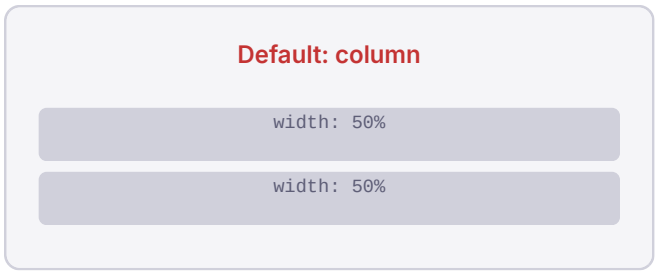
react-pdf supports a subset of CSS. Property names must be camelCase, and several common web CSS properties are not available:

| Web CSS | react-pdf Equivalent |
| --- | --- |
| box-shadow | Not supported – use borderWidth + borderColor |
| text-decoration | textDecoration (underline, line-through) |
| linear-gradient | Not supported – use Svg + Defs |
| display: grid | Not supported – use flexbox |
| font-weight: bold | fontWeight: 700 (numeric or named) |

# Flexbox Layout Surprises

**⚠ Symptom**

Elements stack vertically when you expect them side-by-side. Default flexDirection is "column", not "row".




# Build and Memory Errors

> ⚠ **Symptom**
>
> Build crashes with "JavaScript heap out of memory" or takes extremely long to complete.

Large documents with many images consume significant memory. Node's default heap (typically 2–4 GB, sized from system memory) may not be enough for image-heavy PDFs past ~50 pages.

```bash
# pnpm build still runs pnpm sync first
NODE_OPTIONS=--max-old-space-size=4096 pnpm build
```

- Resize images to actual display size before embedding (2x for retina)
- Use JPEG for photos (quality 80), PNG only for graphics with transparency
- For 50+ page documents, consider splitting into multiple PDF files

# Slow Builds: Find the Culprit

| Symptom | Likely cause | Fix |
|---|---|---|
| Build hangs early | Remote image URLs blocking on the network | Download assets once, embed locally |
| Steady RAM climb | Full-resolution images embedded as-is | Pre-resize to display size, prefer JPEG |
| Slow on every run | Re-registering fonts per render | Register fonts once at module load |

# Debug Workflow

When something looks wrong in your PDF, follow this sequence:

- Simplify: comment out everything except the broken section. Does it still break in isolation?
- Isolate: move the broken component into a standalone single-page document. Render it alone.
- Check wrap: add wrap={false} to containers that should stay together, remove it from containers that need to split.
- Check fonts: temporarily switch to fontFamily: "Helvetica" to rule out font issues.
- Check styles: log style objects to console. Are values what you expect? Are you using camelCase?
- Rebuild incrementally: add sections back one at a time until the break reappears. That's your culprit.

### The Fastest Fix

Most react-pdf layout issues come from three sources: missing wrap=false on elements that should stay together, missing Text wrappers around strings, and flexDirection defaulting to column. Check these three things first before diving deeper.

# Decode the Error

| Error message | What it usually means |
| --- | --- |
| Invalid string child | A raw string sits outside a Text – wrap it. |
| fontFamily ... not registered | The family was never registered – import fonts.ts before render. |
| Cannot read ... of undefined | A style token resolved to undefined – check the import path. |

# Closing the Loop

Chapter 1 started with a simple premise: professional-grade PDF generation is achievable with AI if you apply the right constraints. Every error above has a deterministic solution, and every solution reinforces the architectural patterns we've covered:

| Error | Root Fix | Pattern From |
|---|---|---|
| Text not in <Text> | Wrap all strings | Ch2: Fundamentals |
| Wrong font | Check family/path; register intended weights | Ch2: Fundamentals |
| Content overflow | Shorten/split the fixed source page | Ch2: Fundamentals |
| Orphaned heading | Edit/split the fixed source page | Ch2: Fundamentals |
| Split callout | wrap={false} on outer View | Ch2: Fundamentals |
| Styles ignored | Use camelCase, check support | Ch2: Fundamentals |
| Wrong layout | Set flexDirection: "row" | Ch7: Challenges |
| Out of memory | Resize images, increase heap | Ch11: Troubleshooting |

The debug workflow isn't something you do when things go wrong. It's the last step in the build loop: write, render, export, review, fix. When that loop is fast – and with react-pdf and PNG export, it's very fast – you stop fearing bugs and start shipping.

CHAPTER 12

# Markdown Automation

Markdown authoring with shared components and explicit page breaks.

# The Power of Markdown

Writing content in React components is powerful but tedious. With a Markdown-to-JSX pipeline, you can focus on writing while shared components handle styling; explicit markers still control physical page breaks.

## Why Markdown?

- **Focus on content** – no more JSX syntax errors while writing prose.
- **Portable** – your content lives in standard .md files.
- **AI Friendly** – AI models are excellent at writing and editing Markdown.

# How it Works

The `MarkdownRenderer` component parses your Markdown and maps it to the project's premium component library.

```tsx
import { MarkdownRenderer } from '../components';
import fs from 'fs';
const content = fs.readFileSync('content/chapters/12-markdown-demo.md', 'utf-8');
const body = content.replace(/^---[\s\S]*?---/, '').trim();
const pages = body.split('\n<!-- page-break -->\n');
const page = (index: 0 | 1) => (
<ContentPage sectionTitle="Automation" wrap={false}>
<MarkdownRenderer content={pages[index].trim()} />
</ContentPage>
);
```

### Pro Tip

You can mix and match Markdown with custom JSX for complex layouts that Markdown can't handle.

## Supported Elements

The current parser supports:

- **Headings** (h1, h2, and h3)
- **Body text** with automatic paragraph grouping
- **Bullet lists** using the `BulletList` component
- **Code blocks** with syntax highlighting and a language label
- **Callouts** (Tip, Warning, Info boxes)

> ⚠ **Keep code blocks short**
>
> Fenced blocks render through `CodeBlock`, which uses `wrap={false}` – a block that outgrows the page cannot split, so size each sample to the remaining page height.

## Inline Formatting

You can mix **bold**, *italic*, and `inline code` directly in any paragraph, and the renderer maps each run to the matching text style. Reach for *emphasis* where a word carries weight, and `code` whenever you name a prop, file, or value.

> ⓘ **Round-trip ready**
>
> Because the source is plain Markdown, your AI agent can draft, edit, and review chapters as text – then this pipeline renders them into the same premium components every hand-built page uses.

# Now
# Ship It.

Twelve chapters, from architecture to automation. You have the patterns, the source code, and the templates. The distance between a basic AI-generated PDF and a high-end, professional deliverable is shorter than you think.

1. Structure your project for AI – one file per page, design tokens in one place, components that compose (Ch 3).
2. Define your design language once – colors, typography, spacing, borders. The theme file enforces consistency so you don't have to (Ch 4).
3. Optimize for token budgets – small, focused files mean small, focused prompts that keep AI in its high-attention zone (Ch 5).
4. Export to PNG for visual QA – controlled rasterization gives AI a consistent view of reader-visible layout. The loop is generate, export, review, fix (Ch 9).
5. Iterate past the first draft – premium output takes 2-3 passes. Use the recipes and checklists from Chapter 10 to close the gap.
6. Troubleshoot methodically – check fixed-page overflow, missing Text wrappers, unsupported styles, and column-default flexbox (Ch 11).
7. Automate your workflow – use Markdown for content, shared components for styling, and explicit markers for page breaks (Ch 12).

**The source code for this entire book is included:**

- A design token system (theme.ts) ready to customize.
- A reusable component library – ContentPage, CodeBlock, Table, TipBox, Icons.
- A build and PNG export pipeline.
- AI-optimized reference documentation.

**Clone it. Change the colors. Build your own.**